

**MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ENGINEERING**

New Panvel, Navi Mumbai

Department of Electronics and Telecommunication Engineering



**PROJECT REPORT
LIBRARY MANAGEMENT SYSTEM**

A Java Based Library Management System

Submitted by

Sameer Dinesh Kumar Yadav

Roll No.: 571

Third Year EXTC – A Division

Academic Year

2026–2027

Department of Electronics and Telecommunication Engineering

Pillai College of Engineering

New Panvel, Navi Mumbai

Abstract

The Library Management System is a Java-based application developed to simplify and organize common library operations. The system provides facilities for managing books and students, searching books, issuing books, returning books, and calculating fines for late returns.

The project is implemented using Core Java and follows object-oriented programming principles. The system uses classes such as `Book`, `Student`, `Library`, `FileManager`, `Main`, and `LibraryGUI`. Java Swing is used to provide a graphical user interface for interacting with the system.

File handling is used to maintain the library data so that important information can remain available even after the application is closed. Exception handling is also incorporated to prevent invalid user inputs from causing the application to terminate unexpectedly.

The project demonstrates practical implementation of Java concepts including classes and objects, encapsulation, `ArrayList`, file handling, exception handling, methods, conditional statements, loops, and GUI programming.

Keywords: Java, Library Management System, Core Java, Swing, File Handling, `ArrayList`, OOP, GUI.

Contents

Abstract	1
1 Introduction	7
1.1 Background	7
1.2 Need of the System	7
1.3 Problem Statement	8
1.4 Importance of the Project	8
1.5 Aim	8
1.6 Objectives	8
1.7 Scope	9
2 System Requirements and Feasibility	10
2.1 Hardware Requirements	10
2.2 Software Requirements	10
2.3 Functional Requirements	10
2.4 Non-Functional Requirements	11
2.5 Feasibility Study	11
2.5.1 Technical Feasibility	11
2.5.2 Economic Feasibility	11
2.5.3 Operational Feasibility	12
3 System Design and Methodology	13
3.1 System Overview	13
3.2 System Architecture	13
3.3 Book Management Workflow	14
3.4 Student Management Workflow	14
3.5 Book Issue Workflow	14
3.6 Book Return Workflow	15
4 Object-Oriented Design	16
4.1 Class Diagram	16
4.2 Book Class	16

4.3	Student Class	17
4.4	Library Class	17
4.5	FileManager Class	17
4.6	Main Class	18
4.7	LibraryGUI Class	18
4.8	Object-Oriented Programming Concepts	18
5	Java Implementation Details	19
5.1	Project File Structure	19
5.2	Book Management	19
5.3	Student Management	19
5.4	Book Search	20
5.5	Book Issue	20
5.6	Book Return	20
5.7	Fine Calculation	20
5.8	File Handling	21
5.9	Exception Handling	21
6	Functional Modules and User Interface	22
6.1	Dashboard	22
6.2	Book Collection	22
6.3	Registered Students	23
6.4	Add Book Module	24
6.5	Search Book Module	24
6.6	Issue Book Module	24
6.7	Return Book Module	24
6.8	Exit Module	24
7	Data Storage and Management	25
7.1	ArrayList Storage	25
7.2	Persistent Storage	25
7.3	Book Status Management	25
8	Testing and Results	26
8.1	Testing Methodology	26
8.2	Test Cases	26
8.3	Sample Output	27
8.3.1	Adding a Book	27
8.3.2	Issuing a Book	27
8.3.3	Returning a Book	27

8.3.4 Invalid Input	27
9 Advantages, Limitations and Future Scope	28
9.1 Advantages	28
9.2 Limitations	28
9.3 Future Scope	29
10 Conclusion	30
11 Project Summary	31
References	32

List of Figures

3.1	System Architecture of Library Management System	14
4.1	Class Diagram of Library Management System	16
6.1	Library Management System Dashboard	22
6.2	Book Collection Interface	23
6.3	Registered Students Interface	23

List of Tables

2.1	Hardware Requirements	10
2.2	Software Requirements	10
4.1	OOP Concepts Used in the Project	18
8.1	System Test Cases	26
11.1	Project Summary	31

1. Introduction

1.1 Background

Libraries contain a large amount of information related to books, students, and borrowing activities. Managing these records manually can become time-consuming and may result in errors.

A computerized Library Management System provides a simple way to organize library information. It allows the user to add books, register students, search books, issue books, return books, and maintain the availability status of books.

The proposed system is developed using Java. It provides both a console-based implementation and a graphical user interface using Java Swing.

1.2 Need of the System

A library requires proper management of books and student records. A manual system may involve maintaining registers for book details and student details. Searching and updating these records manually requires additional time.

The proposed system helps in:

- Maintaining book records.
- Maintaining student records.
- Searching books quickly.
- Checking book availability.
- Issuing books.
- Returning books.
- Calculating late-return fines.
- Storing information using file handling.

1.3 Problem Statement

The objective is to develop a Java-based Library Management System that can manage books and students and perform basic library operations through a simple and user-friendly interface.

The system should provide functionality for adding, displaying, searching, issuing, and returning books along with maintaining student information.

1.4 Importance of the Project

The project provides practical implementation of Java programming concepts. It demonstrates how object-oriented programming can be used to create a real-world application.

It also provides experience in:

- Object-oriented programming.
- Data organization using ArrayList.
- File handling.
- Exception handling.
- Java Swing GUI development.
- Modular programming.

1.5 Aim

The aim of this project is to design and develop a simple Java-based Library Management System for managing books, students, issuing, returning, and searching operations.

1.6 Objectives

The major objectives of the project are:

1. To develop a library management application using Java.
2. To store and manage book information.
3. To register and display student information.
4. To provide book searching functionality.

5. To implement book issuing and returning operations.
6. To calculate fines for late book returns.
7. To implement file-based data persistence.
8. To develop a graphical user interface using Java Swing.
9. To demonstrate object-oriented programming concepts.

1.7 Scope

The scope of the project includes basic library management operations.

The system can be used for:

- Adding new books.
- Viewing available books.
- Searching for a particular book.
- Adding students.
- Viewing registered students.
- Issuing books.
- Returning books.
- Calculating fines.
- Maintaining data using files.

The current implementation does not use a MySQL or other external database. The system uses file handling for data storage.

2. System Requirements and Feasibility

2.1 Hardware Requirements

The basic hardware requirements for the project are shown in Table 2.1.

Table 2.1: Hardware Requirements

Component	Requirement
Processor	Intel/AMD processor or equivalent
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Display	Standard monitor
Keyboard	Standard keyboard
Mouse	Standard mouse

2.2 Software Requirements

Table 2.2: Software Requirements

Software	Requirement
Programming Language	Java
Java Version	Java SE 17
IDE	Eclipse IDE
GUI Framework	Java Swing
Operating System	Windows/Linux/macOS
Database	Not required

2.3 Functional Requirements

The functional requirements of the system are:

1. Add a new book.

2. Display all books.
3. Search for a book using Book ID.
4. Add a student.
5. Display registered students.
6. Issue a book.
7. Return a book.
8. Calculate late-return fine.
9. Save library information.
10. Load saved information when the application starts.

2.4 Non-Functional Requirements

The system should satisfy the following non-functional requirements:

- **Usability:** The interface should be simple and easy to understand.
- **Reliability:** The system should handle invalid input without crashing.
- **Maintainability:** The program should be divided into separate classes.
- **Performance:** Basic library operations should execute quickly.
- **Portability:** The Java application can run on systems supporting Java.

2.5 Feasibility Study

2.5.1 Technical Feasibility

The project is technically feasible because it requires only Java and a standard development environment such as Eclipse. No special hardware or external database is required.

2.5.2 Economic Feasibility

The project is economical because it uses Java and freely available development tools. No additional database server or paid software is required.

2.5.3 Operational Feasibility

The system is simple to operate. Users can perform library operations through the console or graphical user interface.

3. System Design and Methodology

3.1 System Overview

The Library Management System consists of multiple Java classes. Each class performs a specific responsibility.

The major components are:

- Book
- Student
- Library
- FileManager
- Main
- LibraryGUI

The `Library` class acts as the main management class and maintains lists of books and students.

3.2 System Architecture

Figure 3.1 represents the overall architecture of the system.

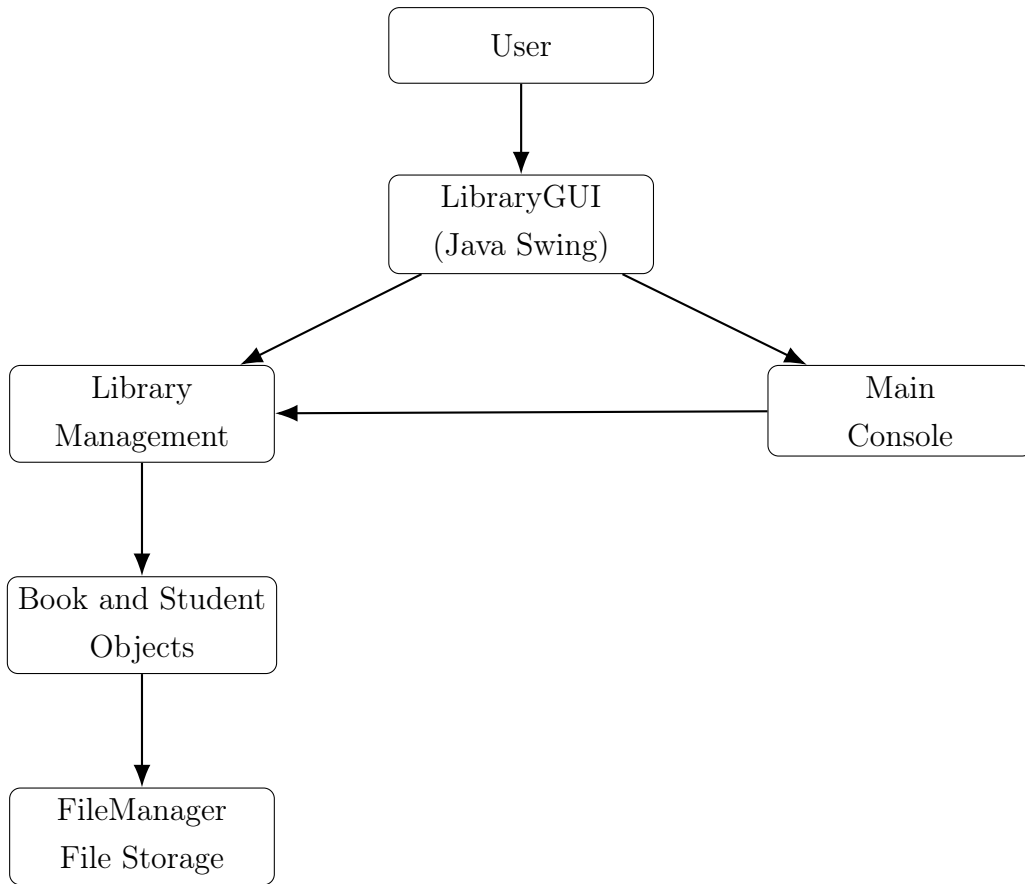


Figure 3.1: System Architecture of Library Management System

3.3 Book Management Workflow

The book management module handles the creation and management of book objects. A book contains a unique Book ID, title, author name, and availability status.

When a new book is created, its availability is initially set to **true**. When the book is issued, the availability becomes **false**. When the book is returned, it becomes **true** again.

3.4 Student Management Workflow

The student module stores the Student ID, student name, and course. Registered students can be displayed through the console or graphical interface.

3.5 Book Issue Workflow

The issue operation first searches for the requested Book ID. If the book exists and is available, its status is changed to issued.

If the book is already issued, the system displays an appropriate message.

3.6 Book Return Workflow

The return operation searches for the requested Book ID. If the book is currently issued, the system changes its status to available.

The project uses a fine of:

$$\text{Fine} = \text{Late Days} \times 5$$

Therefore, the fine is calculated at Rs. 5 per late day.

4. Object-Oriented Design

4.1 Class Diagram

The major classes of the system and their relationships are represented in Figure 4.1.

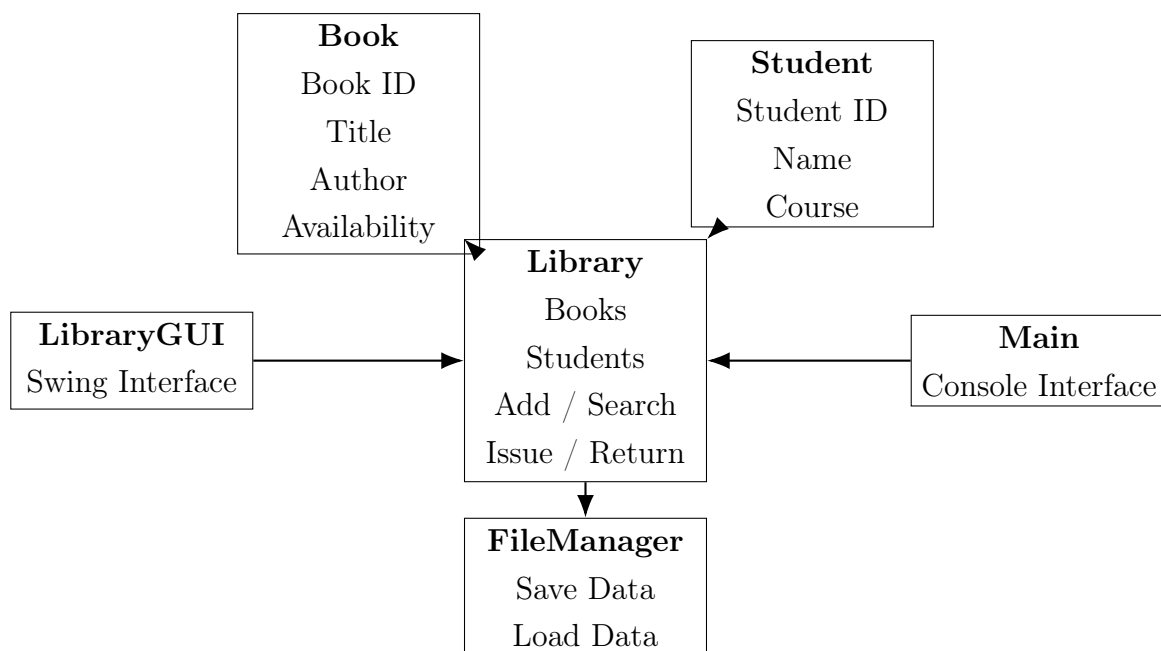


Figure 4.1: Class Diagram of Library Management System

4.2 Book Class

The **Book** class represents an individual book in the library.

The class contains:

- Book ID
- Book title
- Author
- Availability status

Important methods include:

- `issueBook()`
- `returnBook()`
- `isAvailable()`
- `displayBook()`

4.3 Student Class

The `Student` class represents a registered student.

It contains:

- Student ID
- Student name
- Course

The class also provides getter methods and a `displayStudent()` method.

4.4 Library Class

The `Library` class is responsible for managing collections of books and students using `ArrayList`.

The class provides methods for:

- Adding books.
- Adding students.
- Displaying books.
- Displaying students.
- Searching books.
- Issuing books.
- Returning books.

4.5 FileManager Class

The `FileManager` class is responsible for saving and loading library data.

File handling helps preserve important data between program executions.

4.6 Main Class

The `Main` class provides a console-based interface. It displays a menu and accepts user input through the `Scanner` class.

4.7 LibraryGUI Class

The `LibraryGUI` class provides the graphical user interface using Java Swing components.

The GUI provides access to important library operations through buttons, tables, text fields, and dialog boxes.

4.8 Object-Oriented Programming Concepts

Table 4.1: OOP Concepts Used in the Project

Concept	Implementation
Class	Book, Student, Library, FileManager, Main and LibraryGUI
Object	Book and Student objects are created during execution
Encapsulation	Data members are private and accessed using methods
Abstraction	Library methods provide simplified operations
Modularity	Different classes perform different responsibilities

5. Java Implementation Details

5.1 Project File Structure

The project consists of the following Java files:

```
LibraryManagementSystem
|
+-- src
   |
   +-- Book.java
   +-- Student.java
   +-- Library.java
   +-- FileManager.java
   +-- Main.java
   +-- LibraryGUI.java
```

5.2 Book Management

A new book is created using the Book class constructor.

Example:

```
Book book = new Book(bookId, title, author);
library.addBook(book);
```

The book is initially marked as available.

5.3 Student Management

A student is created using the Student class.

```
Student student =
    new Student(studentId, name, course);

library.addStudent(student);
```

The student information can then be displayed using the student list.

5.4 Book Search

The search operation compares the requested Book ID with the Book ID of each book stored in the ArrayList.

If a matching book is found, its details are displayed. Otherwise, the system displays “Book not found.”

5.5 Book Issue

The system first checks whether the requested book exists and whether it is available.

If the book is available, the following operation is performed:

```
book.issueBook();
```

The status of the book then becomes “Issued.”

5.6 Book Return

When a book is returned, the system changes its status back to available.

```
book.returnBook();
```

The number of late days is also accepted.

5.7 Fine Calculation

The project uses a fixed fine of Rs. 5 per late day.

The formula used is:

$$F = D \times 5$$

where:

- F = Fine
- D = Number of late days

For example, if a book is returned 3 days late:

$$F = 3 \times 5 = 15$$

Therefore, the fine is Rs. 15.

5.8 File Handling

File handling is used to maintain information after the application is closed.

The `FileManager` class performs operations such as:

- Saving book information.
- Loading book information.
- Saving student information.
- Loading student information.

The library data is stored in a text file.

5.9 Exception Handling

Exception handling is used to manage invalid user inputs.

For example, if the program expects an integer but the user enters text, the exception is handled instead of allowing the program to terminate.

Example:

```
try {
    choice = scanner.nextInt();
}
catch (Exception e) {
    System.out.println(
        "Invalid input! Please enter a number."
    );
    scanner.nextLine();
}
```

6. Functional Modules and User Interface

6.1 Dashboard

The dashboard provides the main graphical interface of the Library Management System. It provides navigation to important operations such as Books, Students, Add Book, Search Book, Issue Book, Return Book, and Exit.

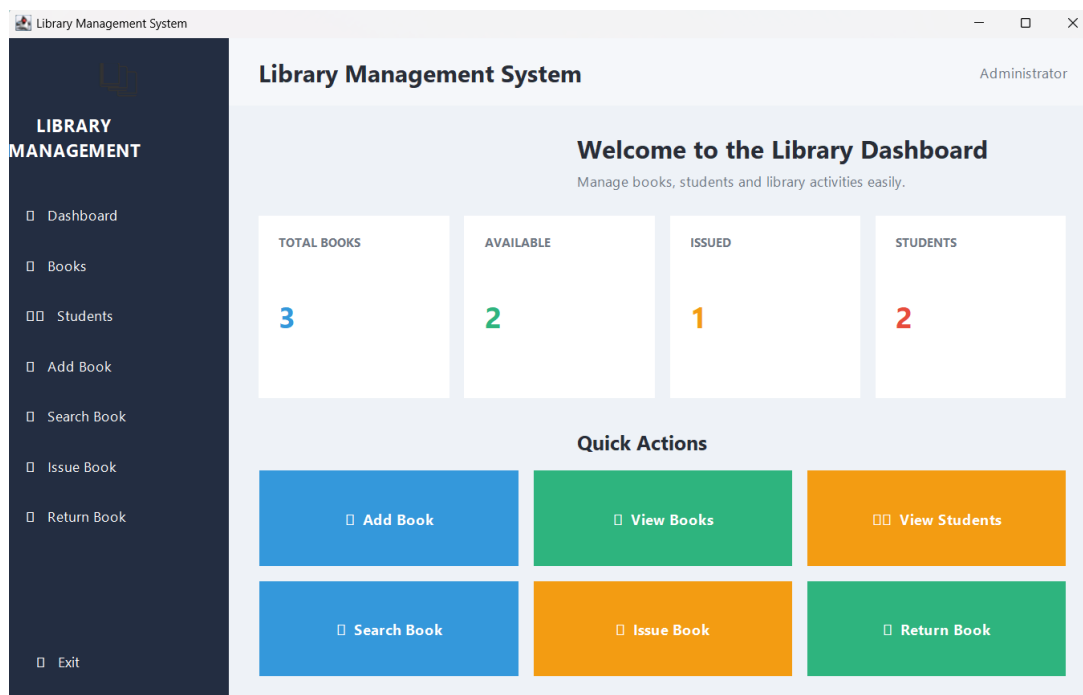


Figure 6.1: Library Management System Dashboard

6.2 Book Collection

The Book Collection interface displays the books available in the library. Important information such as Book ID, title, author, and status can be displayed.

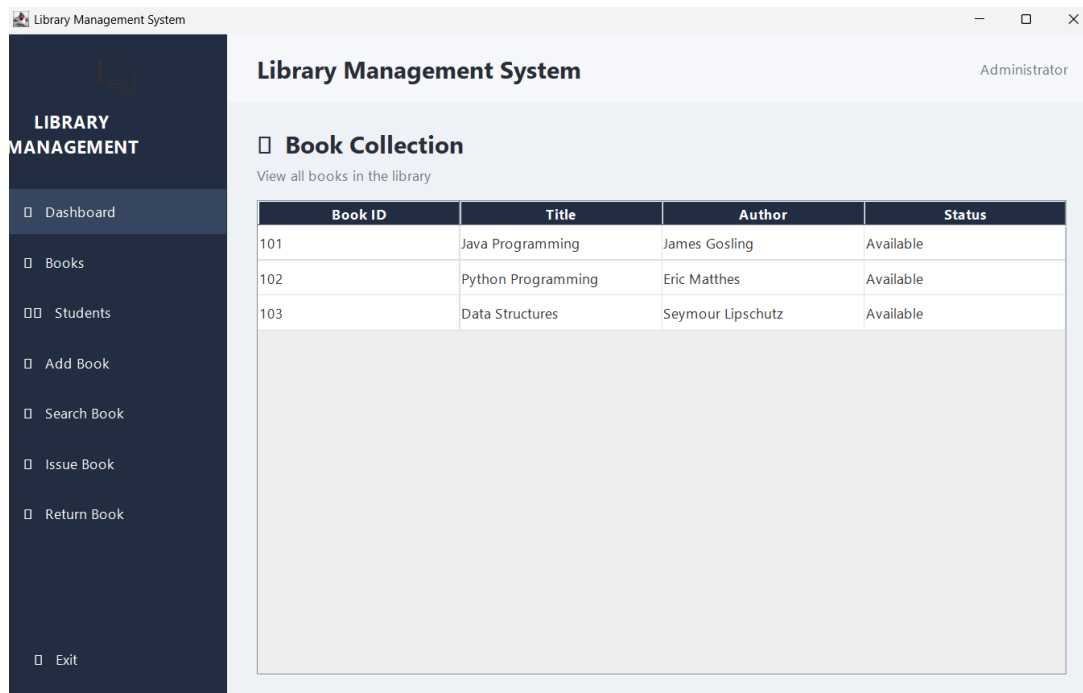


Figure 6.2: Book Collection Interface

6.3 Registered Students

The Registered Students interface displays the students registered in the library system. The student information includes Student ID, student name, and course.

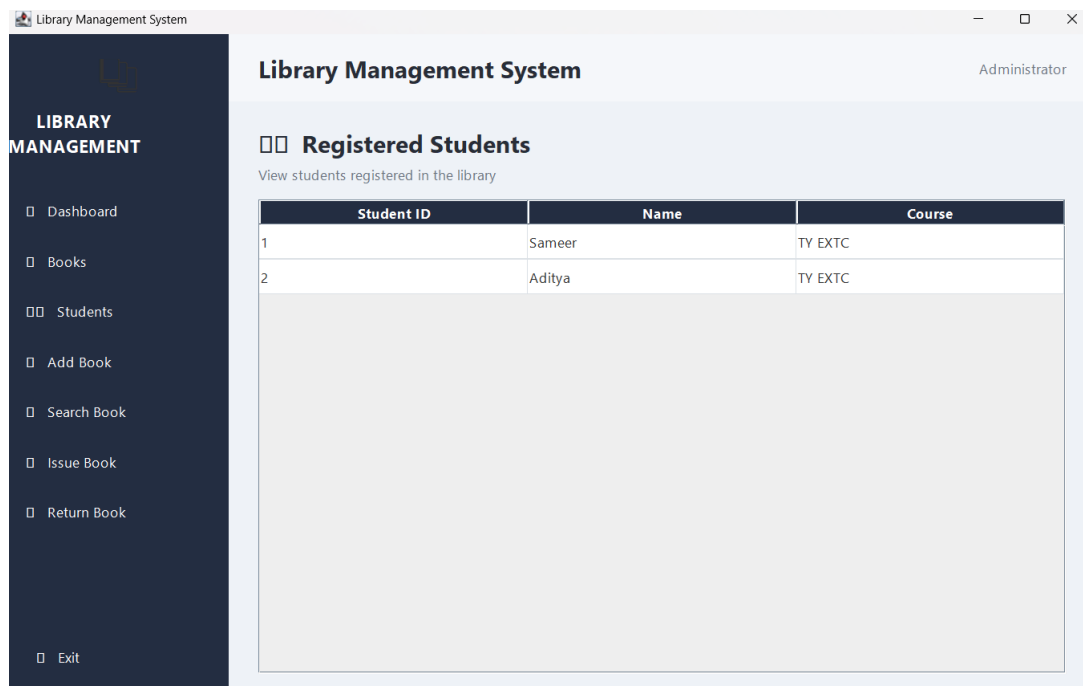


Figure 6.3: Registered Students Interface

6.4 Add Book Module

The Add Book module allows the user to enter:

- Book ID
- Book title
- Author name

After the information is entered, a new Book object is created and added to the library.

6.5 Search Book Module

The Search Book module accepts a Book ID and searches the stored collection. If the book is found, its information is displayed.

6.6 Issue Book Module

The Issue Book module allows a book to be issued by entering its Book ID. The system checks its availability before changing its status.

6.7 Return Book Module

The Return Book module allows the user to enter the Book ID and number of late days.

After returning the book, the system calculates the fine using the fixed rate of Rs. 5 per late day.

6.8 Exit Module

The Exit option closes the application after displaying a suitable message to the user.

7. Data Storage and Management

7.1 ArrayList Storage

The project uses Java's `ArrayList` collection to store books and students during program execution.

The `Library` class maintains:

```
private ArrayList<Book> books;  
private ArrayList<Student> students;
```

This allows multiple book and student objects to be stored dynamically.

7.2 Persistent Storage

File handling is used to store information in a text file.

When the application starts, the `FileManager` loads the stored information. When changes are made, the information can be saved again.

This provides basic persistence without requiring a database.

7.3 Book Status Management

Each `Book` object contains a Boolean availability value.

The two possible states are:

Status	Meaning
Available	Book can be issued
Issued	Book is currently issued

8. Testing and Results

8.1 Testing Methodology

Testing was performed by executing different operations of the system and checking whether the expected result was obtained.

8.2 Test Cases

Table 8.1: System Test Cases

No.	Test Case	Expected Result	Status
1	Add Book	Book is added successfully	Pass
2	View Books	Book list is displayed	Pass
3	Search Book	Matching book details are displayed	Pass
4	Add Student	Student is registered	Pass
5	View Students	Registered students are displayed	Pass
6	Issue Book	Available book becomes issued	Pass
7	Return Book	Issued book becomes available	Pass
8	Fine Calculation	Fine is calculated according to late days	Pass
9	Invalid Input	Error message is displayed	Pass
10	Exit	Application terminates normally	Pass

8.3 Sample Output

8.3.1 Adding a Book

```
Enter Book ID: 101
Enter Book Title: Java Programming
Enter Author Name: Herbert Schildt
```

Book added successfully!

8.3.2 Issuing a Book

```
Enter Book ID to issue: 101
```

Book issued successfully!

8.3.3 Returning a Book

```
Enter Book ID to return: 101
Enter number of late days: 3
```

```
Book returned successfully!
Late Days: 3
Fine: Rs.15
```

8.3.4 Invalid Input

```
Enter your choice: abc
```

Invalid input! Please enter a number.

9. Advantages, Limitations and Future Scope

9.1 Advantages

The major advantages of the system are:

- Simple and user-friendly interface.
- Easy management of books.
- Easy management of students.
- Quick book searching.
- Book availability tracking.
- Book issue and return functionality.
- Automatic fine calculation.
- File-based data persistence.
- Demonstrates practical Java programming concepts.
- No external database is required.

9.2 Limitations

The current system has some limitations:

- It uses text-file storage instead of a database.
- It does not provide a network-based multi-user system.
- Authentication is not implemented.
- Fine calculation is based only on the number of late days.
- Advanced reporting features are not included.

9.3 Future Scope

The system can be further improved by adding:

- MySQL or another database.
- Student and librarian login.
- Password-based authentication.
- Advanced search and filtering.
- Book reservation functionality.
- Email or notification functionality.
- Detailed reports.
- Online library access.
- Automatic backup.
- Multiple-user support.

10. Conclusion

The Library Management System was successfully designed and developed using Java. The project provides basic facilities required for managing books and students in a library.

The system allows the user to add and view books, search for books, register students, issue books, return books, and calculate fines for late returns.

The project also demonstrates important Java concepts such as classes, objects, encapsulation, ArrayList, file handling, exception handling, conditional statements, loops, methods, and Java Swing GUI development.

The use of file handling provides basic data persistence without requiring an external database. The graphical user interface makes the system easier to operate and provides a practical demonstration of the Java application.

Overall, the project provides a simple and effective implementation of a Library Management System while demonstrating the practical application of Core Java and object-oriented programming concepts.

11. Project Summary

Table 11.1: Project Summary

Parameter	Details
Project Title	Library Management System
Programming Language	Java
Java Version	Java SE 17
Development Environment	Eclipse IDE
GUI Technology	Java Swing
Data Structure	ArrayList
Data Storage	Text File
Database	Not Used
Main Modules	Books, Students, Search, Issue, Return
Fine Rate	Rs. 5 per late day
Project Type	Java Application

References

1. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education.
2. Oracle Corporation, *Java Documentation*, <https://docs.oracle.com/en/java/>
3. Oracle Corporation, *Java Swing Documentation*, <https://docs.oracle.com/javase/tutorial/uiswing/>
4. Oracle Corporation, *Java Collections Framework*, <https://docs.oracle.com/javase/tutorial/collections/>
5. Oracle Corporation, *Java File I/O*, <https://docs.oracle.com/javase/tutorial/essential/io/>